

LLM12: LLM Inference — Decoding Strategies & KV-Cache

AUP Learning Cloud — AMD Research

CS290W

May 9, 2026

Two-Stage Inference Pipeline

Prompt Phase (Prefill)

- Process entire prompt in parallel
- Compute-bound: high arithmetic intensity
- Latency: Time to First Token (TTFT)

Token Generation Phase (Decode)

- Generate one token at a time (sequential)
- Memory-bound: low arithmetic intensity
- Throughput: tokens per second

Autoregressive Generation

Autoregressive generation: given a prompt, generate one token at a time.

输入 $X = [x_1, x_2, \dots, x_t]$



自回归 $P(x_{t+1}|x_1, x_2, \dots, x_t) = \text{Softmax}(\mathbf{h}_t \mathbf{W}_{\text{out}} + \mathbf{b}_{\text{out}}),$
 $\mathbf{h}_t \in \mathbb{R}^{d_h} \quad \mathbf{W}_{\text{out}} \in \mathbb{R}^{d_h \times \text{vocab}}$
 $x_{t+1} \sim P(x_{t+1}|x_1, x_2, \dots, x_t).$



下一轮输入 $X = [x_1, \dots, x_t, x_{t+1}]$

Why Decode is Memory-Bound

Key insight: Decode loads the **entire model weights** for just **one token**.

Model Weight Access

Stage	Weights/token
Prefill (s tokens)	$\frac{16h^2L}{s}$
Decode (1 token)	$16h^2L$

Qwen3-0.6B: weights $\approx$ 1.2 GB

At 30 tok/s: $\sim$ 36 GB/s bandwidth!

KV-Cache Access

Stage	KV read/step
Prefill	Write only
Decode	Read all s positions

At $s=8192$: $\sim$ 0.9 GB per decode step

Arithmetic Intensity

Arithmetic intensity = FLOPs / bytes loaded → determines compute vs. memory bottleneck.

Stage	Seq Len	FLOPs	Bytes	Ratio	Intensity
Prefill	128	92B	0.97GB	95	compute
Prefill	8192	13.5T	2.8GB	4779	compute
Decode	128	719M	0.97GB	0.74	memory
Decode	8192	1.6B	2.8GB	0.58	memory

Implication

Prefill processes many tokens in parallel → high intensity → GPU is busy.

Decode processes 1 token → low intensity → GPU is idle, waiting for data.

From Logits to Tokens

The model outputs logits $z \in \mathbb{R}^{|V|}$. How do we pick the next token?

Temperature Scaling

$$\tilde{p}_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

- $T < 1$: sharper $\rightarrow$ more deterministic
- $T > 1$: flatter $\rightarrow$ more random
- $T \rightarrow 0^+$: approaches greedy

Top-K Sampling

Keep only k highest-probability tokens, re-normalize.

Top-P (Nucleus) Sampling

Keep smallest set with cumulative prob $\geq p$.

Typical pipeline: Temperature $\rightarrow$ Top-K $\rightarrow$ Top-P $\rightarrow$ Sample

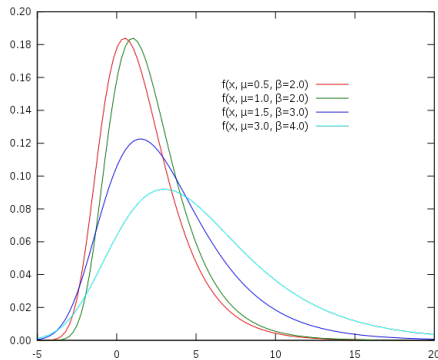
Decoding Strategy Comparison

Demo: 10-token vocabulary, logits = [2.0, 1.5, 1.0, 0.5, 0.0, -0.5, -1.0, -1.5, -2.0, -3.0]

Strategy	Unique tokens sampled (20 trials)
T=0.3 (sharp)	[0, 1, 2]
T=1.0 (default)	[0, 1, 2, 3, 4, 6, 7]
T=2.0 (flat)	[0, 1, 2, 3, 4, 5, 6, 7, 9]
Top-K=3	[0, 1, 2]
Top-P=0.8	[0, 1, 2, 3]
T=0.7 + K=5 + P=0.9	[0, 1, 2]

- Greedy always picks token 0 (highest logit)
- Lower temperature $\rightarrow$ less diversity
- Top-K/P restrict the candidate set

Temperature Effect Visualization



Temperature Control

- **Low T :** High confidence, low diversity (good for factual QA)
- **Medium T :** Balanced creativity and coherence
- **High T :** High diversity, lower quality (good for brainstorming)

Top-K vs Top-P Comparison

Top-K (e.g., $K=3$)

- Sort tokens by probability
- Keep only top K tokens
- Re-normalize probabilities
- Sample from restricted set

Issue

Fixed K may be too restrictive or too loose depending on distribution shape.

Top-P / Nucleus (e.g., $P=0.9$)

- Sort tokens by probability
- Accumulate until sum $\geq P$
- Dynamic candidate set size
- More adaptive than Top-K

Advantage

Automatically adjusts to distribution: few tokens for confident predictions, more for uncertain.

Repetition Penalty Example

Models can produce repetitive text. Three penalty mechanisms:

Repetition Penalty (HuggingFace)

For each previously seen token: divide logit if positive, multiply if negative.

Presence Penalty (OpenAI)

Subtract a constant from any previously seen token's logit.

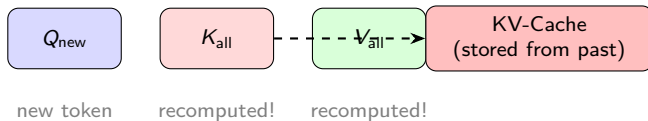
Frequency Penalty (OpenAI)

Subtract value proportional to how many times the token appeared.

Token 0 (appears 3 $\times$)	Original	After penalty
Rep penalty=1.5	3.0	2.00
Freq=0.5 + Pres=0.3	3.0	1.20

What is KV-Cache?

Problem: Without cache, each decode step recomputes K/V for **all** past tokens.



KV-Cache Optimization

Cache past K/V tensors. During decode, only project the **new** token's K/V.

Append to cache: $K = [K_{\text{cache}}; K_{\text{new}}]$, $V = [V_{\text{cache}}; V_{\text{new}}]$

KV-Cache Memory Allocation

Contiguous Allocation

- Pre-allocate fixed-size array
- Simple indexing: `cache[layer][position]`
- **Problem:** Wasted space if sequences vary in length

Paged Attention (vLLM)

- Divide KV-Cache into fixed-size blocks (e.g., 16 tokens)
- Allocate blocks on-demand
- Non-contiguous memory → no internal fragmentation

Benefit

PagedAttention eliminates memory waste from padding → higher batch sizes → better GPU utilization.

KV-Cache Quantization

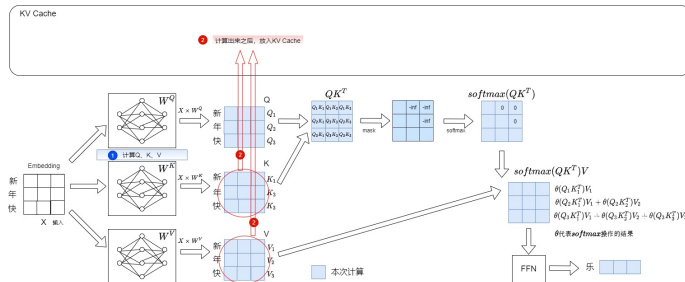
Observation: KV-Cache values don't need full FP16 precision.

Format	Bytes/value	Memory	Quality
FP16 (baseline)	2	100%	Full precision
FP8	1	50%	Minimal loss
INT8	1	50%	Small degradation
INT4	0.5	25%	Noticeable loss

FP8 KV-Cache

Modern GPUs (H100, MI300X) support FP8 natively.
50% memory reduction with negligible quality impact.

KV-Cache Implementation

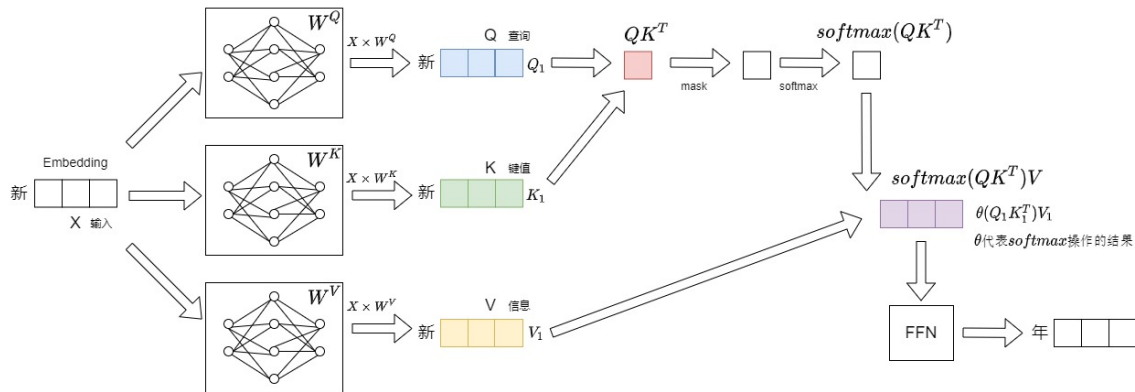


Key Steps

- 1 Compute Q, K, V from input embeddings
- 2 Store K, V in KV Cache for future reuse
- 3 Compute attention using cached K, V
- 4 Append new K, V to cache after each decode step

KV-Cache Step-by-Step Data Flow

Step 1: Input "新" $\rightarrow$ Compute Q, K, V $\rightarrow$ Output "年"

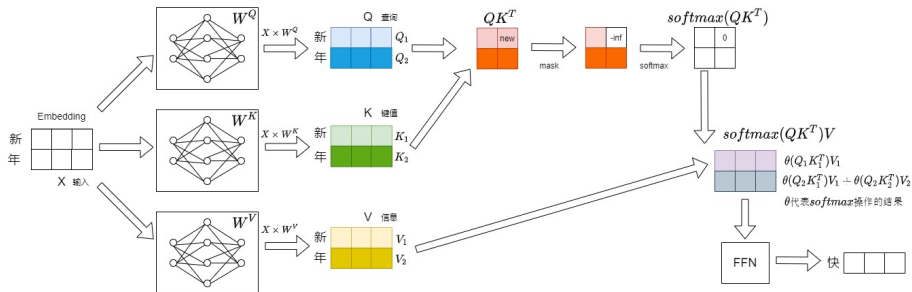


Key Observation

First token: no cache available, compute everything from scratch.

KV-Cache Step 2: Reuse Cached K/V

Step 2: Input "新年" → Reuse cached K/V for "新" → Output "快"

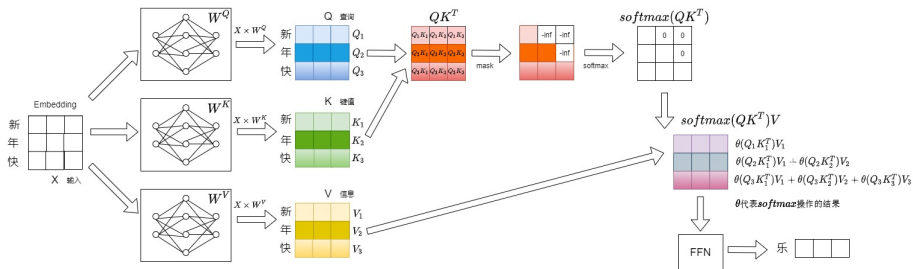


Cache Reuse

- **Red circles:** Repeated computation without cache
- With cache: only compute K/V for new token "年"
- Reuse cached K/V for "新"

KV-Cache Step 3: Cumulative Benefit

Step 3: Input " 新年快" → Reuse cached K/V for " 新年" → Output " 乐"



Growing Benefit

- Cache contains K/V for all previous tokens
- Only compute K/V for the newest token
- Complexity reduced from $O(s^2)$ to $O(s)$

KV-Cache Memory Formula

For a model with GQA (H_{kv} KV heads):

$$\text{KV memory} = 2 \times b \times L \times H_{kv} \times s \times d_{\text{head}} \times \text{dtype_bytes}$$

Seq Length	KV Memory (FP16)	Per Layer
512	67 MB	4 MB
2,048	268 MB	17 MB
8,192	1.0 GB	67 MB
32,768	4.3 GB	268 MB
131,072	17.2 GB	1.0 GB

128K context $\rightarrow$ 17 GB KV-Cache alone!

This motivates GQA, sparse attention, and KV compression.

KV-Cache Memory Calculation Example

Example: Qwen3-0.6B with sequence length 8,192

Per-Layer KV Memory

- $B = 1$ (batch size)
- $H_{kv} = 8$ (KV heads)
- $s = 8,192$ (sequence length)
- $d_{head} = 128$ (head dimension)
- FP16 = 2 bytes per value

Calculation

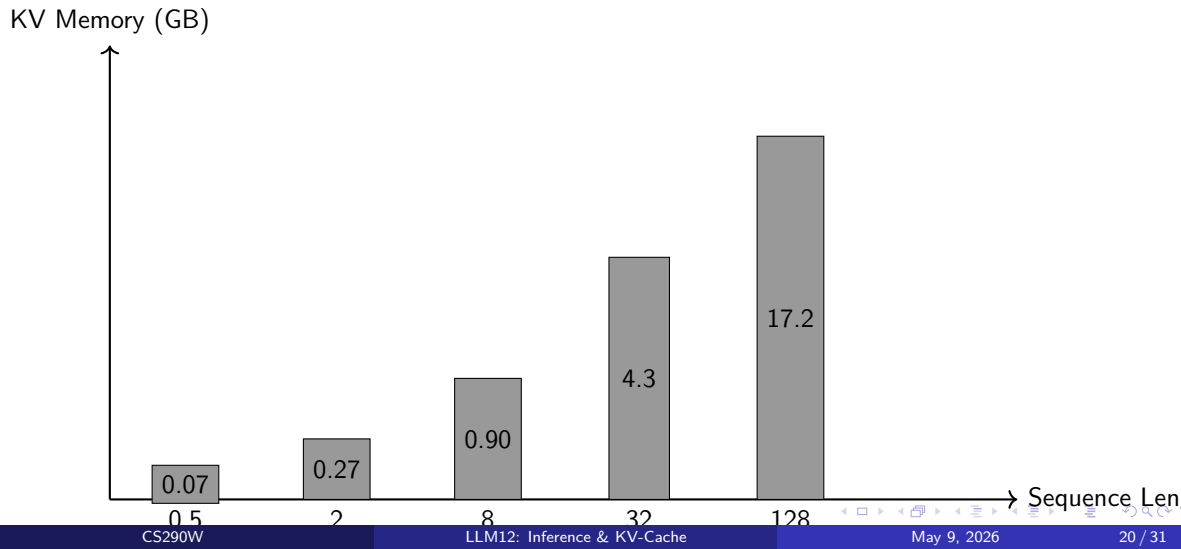
$$\begin{aligned}\text{Per layer} &= 2 \times 1 \times 8 \times 8192 \times 128 \times 2 \\ &= 33,554,432 \text{ bytes} \approx 32 \text{ MB}\end{aligned}$$

$$\text{Total (28 layers)} = 28 \times 32 \text{ MB} \approx 896 \text{ MB}$$

Key Insight

At 8K context, KV-Cache (~ 900 MB) is already comparable to model weights (1.2 GB)!

KV-Cache Growth Pattern



Long-Context Solutions

Technique	Reduction	Key Idea
GQA/MQA	$\times \frac{H_{kv}}{H_q}$	Fewer KV heads
FP8 KV	$\sim 2\times$	Quantize KV cache
Sliding Window	Fixed w	Attend only to last w tokens
Sparse Attention	$\sim 2-5\times$	Select important KV pages
PagedAttention	No waste	Block-based memory mgmt
MLA	$\sim 10\times$	Compress KV to latent space

Batch Serving Pressure

At batch=8, seq=8192: KV-Cache (7.5 GB) **exceeds** model weights (1.2 GB)!

Parameter	Value
Hidden dim	1,024
Layers	28
Q heads	16
KV heads	8
Head dim	128
Vocab size	151,936
Max context	40,960
Total params	596M

Measured Performance

- Prefill: 6 tokens in 467 ms
- Decode: 30 tokens, avg 24 ms/token → 42 tok/s
- KV-Cache: 0.69 MB (28 layers, 6 tokens)

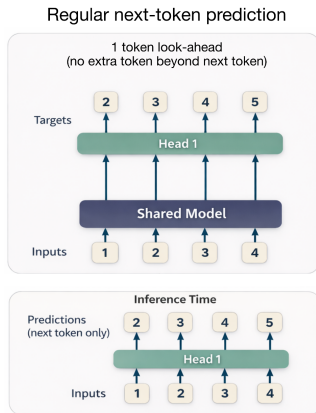
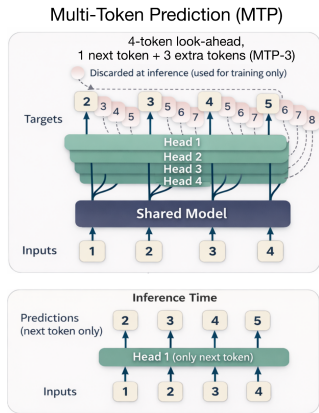
Standard vs Multi-Token Prediction

	Standard	MTP
Training target	$t + 1$ only	$t + 1, t + 2, \dots, t + n$
Prediction heads	1 shared	1 trunk + n heads
Tokens per pass	1	n
Draft model needed	Yes	No
Extra memory	Full model	Small heads only

Key Insight

The model's hidden state encodes enough information to forecast multiple future tokens. MTP is a **learnable skill** — no separate draft model needed.

MTP Architecture



- Each head: `Linear(hidden_dim, vocab_size)` — same as `lm_head`
- Qwen3-0.6B + 4 heads: 596M + 467M params
- In production (DeepSeek-V3): $\sim 1.8\times$ speedup

MTP Inference Flow

One forward pass $\rightarrow n$ tokens!

- 1 Run trunk $\rightarrow$ get hidden state h_t
- 2 Run all n heads in parallel $\rightarrow$ predictions for $t + 1, t + 2, \dots, t + n$
- 3 Accept tokens greedily (or verify with confidence threshold)
- 4 Continue from position $t + n$ (skip $n - 1$ decode steps)

Comparison with Speculative Decoding

- Speculative: needs a separate draft model (extra memory + complexity)
- MTP: model is its own draft (only adds small projection heads)
- Both reduce the number of autoregressive decode steps

Limitation

MTP heads must be trained on multi-token targets.
Randomly initialized heads produce garbage predictions.

MTP Training Objective

Standard Training

- Single prediction head
- Target: next token $t + 1$
- Loss: cross-entropy on $t + 1$

$$\mathcal{L} = -\log P(x_{t+1} | x_{\leq t})$$

MTP Training

- Multiple prediction heads
- Targets: $t + 1, t + 2, \dots, t + n$
- Combined loss across all heads

$$\mathcal{L} = \sum_{i=1}^n w_i \cdot \mathcal{L}_i$$

$$\mathcal{L}_i = -\log P(x_{t+i} | x_{\leq t})$$

Key Difference

MTP heads learn to predict **multiple future tokens** from the same hidden state. This requires the model to encode richer predictive information.

MTP vs Speculative Decoding

Aspect	MTP	Speculative
Draft model	Built-in	Separate small model
Training cost	Higher (multi-head)	Train draft separately
Inference memory	+small heads	+full draft model
Speedup	$\sim 1.5\text{-}2\times$	$\sim 2\text{-}3\times$
Quality	Same as base	May differ (draft bias)
Complexity	Lower	Higher (verify logic)

When to Use Which?

- **MTP**: When you can train/finetune the model, memory-constrained
- **Speculative**: When using frozen models, need maximum speedup

System	Key Features	KV Optimization
vLLM	PagedAttention, continuous batching	Block-based KV mgmt
TensorRT-LLM	Graph optimization, quantization	FP8 KV, page attention
SGLang	RadixAttention, structured output	KV cache reuse
TGI	Multi-device, streaming	Efficient batching

vLLM (Most Popular)

- PagedAttention eliminates memory fragmentation
- Continuous batching: process requests as they arrive
- 2-24 $\times$ throughput improvement over naive serving

Performance Benchmarking

Key Metrics for LLM inference:

Latency Metrics

- **TTFT** (Time to First Token): How long until response starts
- **TPOT** (Time Per Output Token): Average time per token
- **Latency** = $TTFT + (\text{tokens} \times TPOT)$

Throughput Metrics

- **Tokens/sec**: Total tokens generated per second
- **Requests/sec**: Number of completed requests
- **Concurrency**: How many requests handled simultaneously

Trade-off

Higher batch size → better throughput, but higher latency per request.
Optimal batch size depends on SLA requirements.

Summary

- ➊ **Two-stage pipeline:** Prefill (compute-bound) vs Decode (memory-bound)
- ➋ **Arithmetic intensity:** Decode loads entire model for 1 token → very inefficient
- ➌ **Decoding strategies:** Temperature, Top-K, Top-P control diversity vs determinism
- ➍ **KV-Cache:** Essential optimization — cache past K/V to avoid $O(s^2)$ recomputation
- ➎ **GQA:** Halves KV-Cache size with minimal quality loss
- ➏ **Long-context bottleneck:** KV-Cache dominates memory at 32K+ tokens
- ➐ **MTP:** Predict multiple tokens per pass — no separate draft model needed
- ➑ **Advanced techniques:** PagedAttention, Flash Attention, quantization enable efficient serving

Experiment Further

- Train MTP heads and measure real speedup
- Try FP8 KV-Cache quantization
- Compare streaming latency (TTFT vs TPS)
- Benchmark vLLM vs naive serving

Further Reading

- **FlashAttention**: Fast and Memory-Efficient Exact Attention (Dao et al., 2022)
- **vLLM**: Easy LLM Inference Serving (Kwon et al., 2023)
- **DeepSeek-V3**: Multi-Token Prediction (DeepSeek, 2024)
- **GQA**: Training Generalized Multi-Query Models (Ainslie et al., 2023)
- **PagedAttention**: vLLM's memory management technique

Resources

- GitHub: github.com/vllm-project/vllm
- HuggingFace: huggingface.co/docs/transformers
- AMD ROCm: rocm.software